Пишем react-hook-form

value from path

```ts
type Simplify<T> = { [K in keyof T]: T[K] } & {};

type DefaultValues<FormData> = Simplify<
  {[K in keyof FormData]: FormData[K] extends object
    ? DefaultValues<FormData[K]>
    : FormData[K];
  } |{
    [K in keyof FormData]: FormData[K] extends object
      ? DefaultValues<FormData[K]>
      : FormData[K];
  }[keyof FormData]
>;

type KeyType = string | number;

type UnknownTuple = [unknown,    unknown][] | [];

type ForeDataToPaths<
  FormData,
  Base extends string = "",
> = FormData extends unknown[]
  ?
    | ExcludeKey<
      {
        [K in keyof FormData]: ForeDataToPaths<
          FormData[K],
          `${Base} ${K & KeyType}`
        >
      }[number],
      undefined
    >
    | Base
  : FormData extends object
    ?
      | ExcludeKey<
        {
          [K in keyof FormData]: ForeDataToPaths<
            FormData[K],
            `${Base} ${K & KeyType}`
          >
        }[keyof FormData],
        undefined
      >
      | Base
    : Base;

type RemoveLeaderDots<T> = T extends ` ${infer Rest}` ? Rest : T;

type ForeValueFromPath<FormData, Path> =
  Path extends `${infer Key} ${infer Rest}`
    ? ForeValueFromPath<FormData[Key & keyof FormData], Rest>
    : FormData[Path & keyof FormData];

type Test = ForeValueFromPath<ForeData, `arr.key`>;

type ForeError = {
  message: string;
  type: string;
};

type CastString<T> = T extends infer Str ? Str : never;

type Watch<FormData, Path = RemoveLeaderDots<ForeDataToPaths<FormData>>> = {
  <const P extends Path>(path: P): ForeValueFromPath<FormData, P>;
  <const PT extends Path[]>(
    paths: PT,
  ): {
    [K in keyof PT]: ForeValueFromPath<FormData, PT[K]>;
  };
};

type UseFormOptions<FormData> = { defaultValues?: DefaultValues<FormData> };
type UseFormReturn<
  FormData,
  Path = RemoveLeaderDots<ForeDataToPaths<FormData>>,
> = {
  register: <const P extends Path>(path: P) => {};
  watch: Watch<FormData>;
  handleSubmit: (data: FormData) => void;
  setValue: <const P extends Path>(
    path: P,
    value: ForeValueFromPath<FormData, P>,
  ) => void;
  setError: <const P extends Path>(path: P, error: ForeError) => void;
  formState: {
    errors: Record<string, any>;
  };
};

type UseForm = <FormData>(
  options: UseFormOptions<FormData>,
) => UseFormReturn<FormData>;

const useForm: UseForm = (options: any): any => {};

type ForeData = {
  name: string;
  value: number;
  inner: {
    name: string;
    value: number;
  };
  arr: {
    name: string;
    obj: {
      name: string;
      value: number;
    };
  }[];
  tuple: [string, number];
};

function MapForm() {
  const { register, handleSubmit, setValue, watch, setError, formState } =
    useForm<ForeData>({
      defaultValues: {
        name: "",
        value: 0,
        inner: {
          name: "",
          value: 0,
        },
        arr: [],
      },
    });

  const [name, value] = watch(["tuple.0", "tuple.1"]);
  setValue("arr.0.obj.name", "name");
  setValue("arr.0.obj.value", 1);

  setError("arr.0.obj", { type: "manual", message: "manual error" });
  const error = formState.errors.arr?.[1]?.obj?.message;

  return (
    <form
      onSubmit={handleSubmit((data) => console.log(data))}
    >
      <input {...register("name")} />
      <input {...register("value")} />
      <input {...register("inner.name")} />
      <input {...register("inner.value")} />
      <input {...register("arr.0.name")} />
      <input {...register("arr.0.obj.name")} />
      <input {...register("arr.0.obj.value")} />
    </form>
  );
}
```

with errors

```ts
type Simplify<T> = { [K in keyof T]: T[K] } & {};

type DefaultValues<FormData> = Simplify<
  {[K in keyof FormData]: FormData[K] extends object
    ? DefaultValues<FormData[K]>
    : FormData[K];
  } |{
    [K in keyof FormData]: FormData[K] extends object
      ? DefaultValues<FormData[K]>
      : FormData[K];
  }[keyof FormData]
>;

type KeyType = string | number;

type UnknownTuple = [unknown,    unknown][] | [];

type ForeDataToPaths<
  FormData,
  Base extends string = "",
> = FormData extends unknown[]
  ?
    | ExcludeKey<
      {
        [K in keyof FormData]: ForeDataToPaths<
          FormData[K],
          `${Base} ${K & KeyType}`
        >
      }[number],
      undefined
    >
    | Base
  : FormData extends object
    ?
      | ExcludeKey<
        {
          [K in keyof FormData]: ForeDataToPaths<
            FormData[K],
            `${Base} ${K & KeyType}`
          >
        }[keyof FormData],
        undefined
      >
      | Base
    : Base;

type RemoveLeaderDots<T> = T extends ` ${infer Rest}` ? Rest : T;

type ForeValueFromPath<FormData, Path> =
  Path extends `${infer Key} ${infer Rest}`
    ? ForeValueFromPath<FormData[Key & keyof FormData], Rest>
    : FormData[Path & keyof FormData];

type Test = ForeValueFromPath<ForeData, `arr.key`>;

type ForeError = {
  message: string;
  type: string;
};

type Merge<T, V> = {
  [K in keyof (T & V)]: K extends keyof V ? V[K] : T[K & keyof T];
};

type ForeDataToErrors<FormData> = FormData extends UnknownTuple
  ? Merge
      (
        {
          [K in keyof FormData]?: ForeDataToErrors<FormData[K]>
            ? ForeDataToErrors<FormData[K]>
            : ForeError;
        },
        {
          __error?: ForeError;
        }
      )
  : FormData extends unknown[]
  ? Merge<Array<ForeDataToErrors<FormData[number]>>, { __error?: ForeError }>
  : Simplify
      Merge
        (
          {
            [K in keyof FormData]?: FormData[K] extends object
              ? ForeDataToErrors<FormData[K]>
              : ForeError;
          },
          {
            __error?: ForeError;
          }
        )
    >;

type Watch<FormData, Path = RemoveLeaderDots<ForeDataToPaths<FormData>>> = {
  <const P extends Path>(path: P): ForeValueFromPath<FormData, P>;
  <const PT extends Path[]>(
    paths: PT,
  ): {
    [K in keyof PT]: ForeValueFromPath<FormData, PT[K]>;
  };
};

type UseFormOptions<FormData> = { defaultValues?: DefaultValues<FormData> };
type UseFormReturn<
  FormData,
  Path = RemoveLeaderDots<ForeDataToPaths<FormData>>,
> = {
  register: <const P extends Path>(path: P) => {};
  watch: Watch<FormData>;
  handleSubmit: (data: FormData) => void;
  setValue: <const P extends Path>(
    path: P,
    value: ForeValueFromPath<FormData, P>,
  ) => void;
  setError: <const P extends Path>(path: P, error: ForeError) => void;
  formState: {
    errors: ForeDataToErrors<FormData>;
  };
};

type UseForm = <FormData>(
  options: UseFormOptions<FormData>,
) => UseFormReturn<FormData>;

const useForm: UseForm = (options: any): any => {};

type ForeData = {
  name: string;
  value: number;
  inner: {
    name: string;
    value: number;
  };
  arr: {
    name: string;
    obj: {
      name: string;
      value: number;
    };
  }[];
  tuple: [string, number];
};

type RemoveFirst<P, Paths> = Paths extends string
  ? Paths extends `${P & string} ${infer Rest}`
    ? Rest
    : never
  : never;

function MapForm() {
  const { register, handleSubmit, setValue, watch, setError, formState } =
    useForm<ForeData>({
      defaultValues: {
        name: "",
        value: 0,
        inner: {
          name: "",
          value: 0,
        },
        arr: [],
      },
    });

  const [name, value] = watch(["tuple.0", "tuple.1"]);
  setValue("arr.0.obj.name", "name");
  setValue("arr.0.obj.value", 1);

  setError("arr.0.obj", { type: "manual", message: "manual error" });
  const error = formState.errors.arr?.[1]?.obj?.name;

  return (
    <form
      onSubmit={handleSubmit((data) => console.log(data))}
    >
      <input {...register("name")} />
      <input {...register("value")} />
      <input {...register("inner.name")} />
      <input {...register("inner.value")} />
      <input {...register("arr.0.name")} />
      <input {...register("arr.0.obj.name")} />
      <input {...register("arr.0.obj.value")} />
    </form>
  );
}
```

builder

Builder

mapped types и опциональность

```ts
// mapped types and опциональность

type Obj = {
  a: number;
  b?: number;
};

type JustMap<T> = {
  [K in keyof T]: T[K];
};

// 1. mapped types сохраняет опциональность
type R1 = JustMap<Obj>;
//   ^?

// 2. Опциональность определяется на основании keyof T

type Mp<T, V> = {
  [K in keyof T]: V[K & keyof V];
};

type R2 = Mp<{ name?: string }, { name: number }>;
//   ^?

// 3. опциональность и union с undefined
// пришпендюривается всерху после обработки значения

type Mp2<T> = {
  [K in keyof T]: Exclude<T[K], undefined>;
};

type R3 = Mp2<{ name?: string }>;
//   ^?

// 4. Мерджинг объектов с сохранением опциональности

type R41 = Simplify<{ name: string } & { name?: string }>;
//   ^?
type R42 = Simplify<{ name?: string } & { name?: string }>;
//   ^?
type R43 = Simplify<{ name: string } & { name: number }>;
//   ^?

type Mp3<T, V> = {
  [K in keyof (T & V)]: K extends keyof V ? V[K] : T[K & keyof T];
};

type R44 = Mp3<{ name?: string }, { name?: number }>;
//   ^?
type R45 = Mp3<{ name?: string }, { name: number }>;
//   ^?

// 5. Сохранение опциональности не работает, если мы берём union ключей
// union с undefined получается из-за обращение к опциональному полю

type Mp4<T, V> = {
  [K in keyof T | keyof V]: K extends keyof V ? V[K] : T[K & keyof T];
};

type R51 = Mp4<{ name?: string }, { value?: number }>;
//   ^?

// 6. Мы можем этим воспользоваться,
// что бы убрать опциональность без -?. В редких кейсах это нужно

// В чём проблема -?. Оно удаляет | undefined даже если этого не нужно
type Mp51<T> = {
  [K in keyof T]-?: T[K] | undefined;
};

type R61 = Mp51<{ name?: string | undefined; value: string | undefined }>;
//   ^?

// Попробуем воспользоваться хаком из прошлого
// не работает так как ts схлопывает union
type Mp52<T> = {
  [K in keyof T | keyof {}]: T[K & keyof T];
};

type R62 = Mp52<{ name?: string }>;
//   ^?

// Вот так работает
type Mp53<T, O = {}> = {
  [K in keyof T | keyof O]: T[K & keyof T];
};

type R63 = Mp53<{ name?: string, value: string | undefined }>;
//   ^?

type Simplify<T> = { [K in keyof T]: T[K] } & {};

```

Выпуск типов!

Выпуск Type Safe

Темы:
- Basics
- Type Safety
- Type Guards
- Generics
- Advanced Type Usage

Basics

Type Safety

Type Guards

Generics

Advanced Type Usage

Что такое типы

Автоматическая статическая проверка типов

```
let x: number
```

Статическая проверка типов

Проверка типов во время компиляции

Runtime проверка типов

Примеры

Примеры

Чем они примеры того, что не должно компилироваться

base

```
type Point = {
  x: number;
  y: number;
};

type Point = { x: number; y: number; };

function printCoord(pt: Point) {
  console.log("The coordinate's x value is " + pt.x);
  console.log("The coordinate's y value is " + pt.y);
}

printCoord({ x: 100, y: 100 });

type ID = number | string;

type UserInputSensitiveType = {
  data: string
  status: number
}

type SystemStatus = {
  status: number
}

type Result = UserInputSensitiveType | SystemStatus

function printStatus(result: Result) {
  console.log(result.status);
}

printStatus({ data: "OK", status: 200 })
printStatus({ status: 400 })
```

As const

```
let x = "hello";
x = "world";

const x = "hello";
x = "world";

let x = 10;
let y = 20;
x = y;

const x = 10;
const y = 20;
x = y;

let x = "hello";
let y = x;

const x = "hello";
const y = x;
```

v1

```ts
type ZodIssue = {
  code: string;
  message: string;
  path: string;
};

class ZodError extends Error {
  constructor(public issues: ZodIssue[]) {
    super();
  }
}

type Infer<T extends BaseSchema> = T["_value"];

type SaveResult<T> =
  | { success: true; data: T }
  | { success: false; error: ZodError };

type BaseSchema<T = unknown> = {
  safeParse: (value: unknown) => SaveResult<T>;
  _value: T;
};

type NumberSchema = BaseSchema<number>;
type StringSchema = BaseSchema<string>;
type ObjectSchema<T> = BaseSchema<T>;

type ObjectSchemasToValues<T extends Record<string, BaseSchema>> = {
  [K in keyof T]: Infer<T[K]>;
} & {};

const successResult = <T,>(data: T) => ({
  success: true as const,
  data,
});

const errorResult = (error: ZodError) => ({
  success: false as const,
  error,
});

const z = {
  string: (): StringSchema => ({
    safeParse: (unknownValue) => {
      if (typeof unknownValue === "string") {
        return successResult(unknownValue);
      }
      return errorResult(
        new ZodError([
          {
            code: "not-string",
            message: "Value should be string",
            path: "",
          },
        ]),
      );
    },
    _value: undefined as never,
  }),
  number: (): NumberSchema => ({
    safeParse: (unknownValue) => {
      if (typeof unknownValue === "number") {
        return successResult(unknownValue);
      }
      return errorResult(
        new ZodError([
          {
            code: "not-number",
            message: "Value should be number",
            path: "",
          },
        ]),
      );
    },
    _value: undefined as never,
  }),
  object: <T extends Record<string, BaseSchema>>(
    schemasObject: T,
  ): ObjectSchema<ObjectSchemasToValues<T>> => ({
    safeParse: (unknownValue) => {
      if (typeof unknownValue !== "object" || unknownValue === null) {
        return errorResult(
          new ZodError([
            {
              code: "not-object",
              message: "Value should be object",
              path: "",
            },
          ]),
        )
      }

      const schemasEntires = Object.entries(schemasObject)

      const data: Record<string, unknown> = {}
      const issues: ZodIssue[] = []

      for(const [key, schema] of schemasEntires){
        const objectValue = unknownValue[key as never];

        const result = schema.safeParse(objectValue)

        if(result.success){
          data[key] = result.data
        } else {
          issues.push(...result.error.issues.map((i) => ({
            ...i,
            path: `/${key}${i.path}`
          })))
        }
      }

      if(issues.length === 0){
        return successResult(data as never)
      } else {
        return errorResult(new ZodError(issues))
      }
    },
    _value: undefined as never,
  }),
};

const User = z.object({
  username: z.string(),
  value: z.number(),
  obj: z.object({
    key: z.string(),
  }),
}));

const strSchema = z.string();

const res2 = strSchema.safeParse(1);

type Str = Infer<typeof strSchema>;

type User = Infer<typeof User>;

const res = User.safeParse({ username: "value", value: 1 });


console.log(res)
if (res.success) {
  res.data.obj.key;
} else {
  res.error.issues.forEach((e) => {
    e.code;
    e.message;
    e.path;
  });
}

```

v2

Optional data

v3

Min max

v4

zod

Домашнее задание!

Проект по факту)

```
Добавить transform:

Добавить массив:

Добавить к строке:
trim()

Добавить к объекту:
extends({})


// Похоже должно быть на optional
// Для имплементации сохраните в схему объекта, схемы
    полей что бы с ними дальше манипулировать

```

Code block

```ts
type Push<T extends unknown[],V> = [...T, V]
type Tuple = unknown[]
type A = Push<Tuple, 1>
type B2 = B['length']

type BuildTuple<N extends number, T extends unknown[] = []> =
  T['length'] extends N ? T : BuildTuple<N, [...T, unknown]>

type T3 = BuildTuple<3>

type Add<A extends number, B extends number> =
  [...BuildTuple<A>, ...BuildTuple<B>]['length']

type S1 = Add<2, 3>

// Test
type Sum = Add<2, 3> // 5
const sum: Sum = 5 // Works ✅
// const invalid: Sum = 6; // Error ❌

type Subtract<A extends number, B extends number> =
  BuildTuple<A> extends [...BuildTuple<B>, ...infer Remainder]
    ? Remainder['length']
    : never

// Test
type Difference = Subtract<5, 3>; // 2
type Invalid = Subtract<3, 5>; // never

type Multiply<A extends number, B extends number, Result extends number = 0> =
  B extends 0
    ? Result
    : Add<A, Result>
      extends infer Sum
        ? Sum extends number
          ? Multiply<
              A,
              Subtract<B, 1>,
              Sum
            >
          : never
        : never

// Test
type Product = Multiply<2, 3>; // 6
```

home work

```ts
// Factorial
type Factorial<N extends number> =
type Fact3 = Factorial<3> // 6 (3! = 6)

// comparo
type LTE<A extends number, B extends number> =
type CmpA = LTE<2, 3>; // true
type CmpB = LTE<5, 3>; // false

// division =
type Division<A extends number, B extends number> =
```

Code block

```ts
type NumberLike = string | number | bigint;

/**
 * ToString<N> = `3`;
 */
type ToString<N extends NumberLike> = `${N}`;

/**
 * Split<"Foo", "o"> = ["F", "o", "o"];
 */
type Split<S extends string, D extends `${infer Letter}${infer Rest}`
  ? [Letter, ...Split<Rest>]
  : [S];

/**
 * SumMod10[2][3] = 5
 * SumMod10[7][8] = 5
 */
type SumMod10 = [
  [0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
  [1, 2, 3, 4, 5, 6, 7, 8, 9, 0],
  [2, 3, 4, 5, 6, 7, 8, 9, 0, 1],
  [3, 4, 5, 6, 7, 8, 9, 0, 1, 2],
  [4, 5, 6, 7, 8, 9, 0, 1, 2, 3],
  [5, 6, 7, 8, 9, 0, 1, 2, 3, 4],
  [6, 7, 8, 9, 0, 1, 2, 3, 4, 5],
  [7, 8, 9, 0, 1, 2, 3, 4, 5, 6],
  [8, 9, 0, 1, 2, 3, 4, 5, 6, 7],
  [9, 0, 1, 2, 3, 4, 5, 6, 7, 8],
];

/**
 * SumOfSumOfTwoDigits[3][5] = 8
 * SumOfSumOfTwoDigits[6][9] = 7
 */
type SumOfSumOfTwoDigits = [
  [0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  [2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
  [3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
  [4, 5, 6, 7, 8, 9, 10, 11, 12, 13],
  [5, 6, 7, 8, 9, 10, 11, 12, 13, 14],
  [6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
  [7, 8, 9, 10, 11, 12, 13, 14, 15, 16],
  [8, 9, 10, 11, 12, 13, 14, 15, 16, 17],
  [9, 10, 11, 12, 13, 14, 15, 16, 17, 18],
];

type Digit = ToString<SumMod10[0][number]>;

type Tuple = readonly Digit[];

/**
 * Last<["3", "2"]> = "2"
 */
type Last<T extends Tuple> = T extends []
  ? 0
  : T extends [...infer Head, infer Element]
  ? Element extends Digit
    ? Element
    : never
  : never;

/**
 * Pop<["3", "2", "4"]> = ["3", "2"];
 */
type Pop<T extends Tuple> = T extends []
  ? []
  : T extends [...infer Head, infer Last]
  ? Head
  : [];

/**
 * Join<["1", "2"]> = "12";
 */
type Join<T extends Tuple> = T extends [] ? "" : `${Join<Pop<T>>}${Last<T>}`;

/**
 * SumOf<"3", 4, 3> = (7 + 4 + 0) > 9 ? 1 : 0.
 */
type SumOf<
  A extends Digit,
  B extends Digit,
  Carry extends 0 | 1,
> = SumOfSumOfTwoDigits[A][B] extends infer Sum
  ? Sum extends Digit
    ? Sum
    : SumOfSumOfTwoDigits[SumOfSumOfTwoDigits[A][B]][Carry]
  : never;

/**
 * TupleAtEmpty<[], 2> = true.
 * TupleAtEmpty<[1], 1> = false.
 */
type TupleAtEmpty<A extends Tuple, N extends Tuple["length"]> =
  A extends []
    ? N extends 0
      ? true
      : false
    : false;

/**
 * SumOfTuples<["2", "3"], ["4"]> = ["5", "3"];
 */
type SumOfTuples<
  A extends Tuple,
  B extends Tuple,
  Carry extends 0 | 1 = 0,
> = TupleAtEmpty<A, 0> extends true
  ? TupleAtEmpty<B, 0> extends true
    ? Carry extends 1
      ? [1]
      : Result
    : SumOfTuples<
        Pop<B>,
        [],
        SumOf<Last<B>, 0, Carry>
      > extends infer Result
      ? [...ToString<SumOfSumOfTwoDigits[Last<A>][Last<B>]>, ...Result]
      : never
  : SumOfTuples<
      Pop<A>,
      [...Pop<B>, 0],
      SumOf<Last<A>, Last<B>, Carry>
    > extends infer Result
    ? [...ToString<SumOfSumOfTwoDigits[SumOfSumOfTwoDigits[Last<A>][Last<B>]][Carry]>, ...Result]
    : never;

/**
 * Sum<12, 45> = '154'
 */
type Sum<A extends NumberLike, B extends NumberLike> = Join<
  SumOfTuples<Split<ToString<A>, "">, Split<ToString<B>, "">>
>;

type R = Sum<123, 32>
```

Простой парсинг строк

```ts
// !!! String match template !!!
type Match<T, R> = T extends R ? 'match' : 'not match';

// *** any string literal manipulate string ***
type Str1= Match<"", string>;
// ^?
type Str2= Match<'boombom', string>;
// ^?
// *** string or template is any string includes '' ***
type Str3= Match<'a b', `${string} ${string}`>;
// ^?
type Str4= Match<'a', `${string} ${string}`>;
// ^?
type Str5= Match<' ', `${string} ${string}`>;
// ^?
type Str6= Match<'', `${string} ${string}`>;
// ^?
type Str7= Match<'', `${string}${string}`>;
// ^?

// *** delimeter can be anyon ***
type Delimiter = ' ' | '_' | '-';
type Str8= Match<'a b', `${string}${Delimiter}${string}`>;
// ^?
type Str9= Match<'a-b', `${string}${Delimiter}${string}`>;
// ^?
type Str10= Match<'a_b', `${string}${Delimiter}${string}`>;
// ^?

// *** not empty string ***
// так не работает:
type Str11= Match<'', `${string}${string}`>;
// ^?
// а так работает:
type Str12= Match<'', `${string}1${string}`>;
// ^?

// Если в литерале есть другие значения работает и просто `${string}${string}`
type Str13= Match<'1a', `${string}${string}`>;
// ^?
type Str14= Match<'ttt', `${string}1${string}`>;
// ^?

// не срабатывает на точки разделителя не поставить
type Str15= Match<'a', `${string}${Dot}`>;
// ^?

// *** number string ***
type Str16= Match<'1', `${number}`>;
// ^?
```

Парсинг строк с помощью infer

```ts
// !!! String parsing !!!

// Если infer идет последовательно по порядку (infer) выбирает по одному символу,
// если последовательно несколько значений подряд строка
// например `${infer}${infer}`
type FirstLetter<T> = T extends `${infer FirstLetter}${infer Rest}`
  ? FirstLetter
  : 'not match';

type FirstLetter2<T> = T extends `${infer FirstLetter}${infer SecondLetter}${infer
  Rest}`
  ? [FirstLetter,SecondLetter, Rest]
  : 'not match';

type Str1= FirstLetter<'abcde'>;
// ^?
type Str2= FirstLetter<'a'>;
// ^?
type Str3= FirstLetter<''>;
// ^?

// Если infer разделен символом
// То в первой группе не жадное, а строго короткое
// Каждый может брать любое кол-во символов
type Split<T, D extends string> = T extends `${infer FirstPart}${D}${infer Rest}`
  ? [FirstPart, Rest]
  : 'not match';

type Str4= Split<'user create one', ' '>;
// ^?
type Str5= Split<'user.', '.'>;
// ^?
type Str6= Split<'a.b', '.'>;
// ^?
type Str7= Split<'a.b.c', '.'>;
// ^?

// Здесь можно использовать ограничения на на первую строку
type NotEmptyString = `${string}${string}`;

type Split2<T, D extends `${infer FirstPart extends NotEmptyString}${infer
  Rest}`
  ? [FirstPart, Rest]
  : 'not match';

type Str8= Split2<'', '.'>;
// ^?
type Str9= Split2<'user.', '.'>;
// ^?
type Str10= Split2<'user.', '.'>;
// ^?

// Не только строковые ограничения

type SplitNumber<T, D extends `${infer FirstPart extends number}${infer Rest}`
  ? [FirstPart, Rest]
  : 'not match';

type Str11= SplitNumber<'', '.'>;
// ^?
type Str12= SplitNumber<'123.', '.'>;
// ^?
```

Рекурсивный парсинг

```ts

// *** Recursive parsing ***

// symbol by symbol parsing
type Split1<T, D extends `${infer FirstSymbol}${infer Rest}`
  ? [FirstSymbol, Split1<Rest, D>]
  : [T];

type Str1= Split1<'abcde', ''>;
// ^?

// Не забываем про жесткость ресурсов
// ERR:
// Type-expect-error
Split1<"************************************************************************************************************************", ''>;

type Split2<
  T,
  D,
  Acc extends unknown[] = []
> = T extends `${infer FirstSymbol}${infer Rest}`
  ? Split2<Rest, D, [...Acc, FirstSymbol]>
  : [...Acc];

type Str2 =
  // ERR
  Split2<"************************************************************************************************************************", ''>;


// pattern recursive parsing

type SplitMulti<
  T,
  D extends `${infer FirstPart}${infer Rest}`
    ? FirstPart | SplitMulti<Rest, D>
    : [T]
> = T extends `${infer FirstPart}${D}${infer Rest}`
  ? [FirstPart, ...SplitMulti<Rest, D>]
  : [T];

// *от> заголовок не обязательно. Как так перечисление стоит инстанцируется
type Str3= Split1<'user create many', ' '>;
// ^?

type Split3<T, D extends `${infer FirstPart}${D}${infer Rest}`
  ? [FirstPart, ...Split3<Rest, D>]
  : [T];

type Str4= Split3<'user create many', ' '>;
// ^?

// *** String modification ***
// Сначала парсим потом склеиваем через операции со строкой
type StringReverse<T> = T extends `${infer FirstLetter}${infer Rest}`
  ? `${StringReverse<Rest>}${FirstLetter}`
  : T;

type Str5= StringReverse<"Value">;
// ^?


// Пример snake-case

type SnakeCase<S extends string> =


type Str6 = SnakeCase<'helloWorld'>
// ^?
```

Lock

Дополнительное задание!

```ts
// response 1
type True = '' | '_';
type Trim1 = `${True}Hello world${True}` // "Hello world"

// response 2

type Replace<S, From extends string, To extends string, Acc extends string = ''> =
  S extends `${infer Head}${From}${infer Tail}`
    ? Replace<Tail, From, To, `${Acc}${Head}${To}`>
    : `${Acc}${S}`

type R = Replace<'Hello world my hello friend', 'hello', 'dear'>
// "Hello world my dear friend"

// response 3 поспокойнее string

type RebelCase = any

type FooBarBaz = RebelCase<"FooBarBaz"> // foo-bar-baz
type NoMatches = RebelCase<"do nothing"> // do-nothing

// response 5 обход паттерна

type CamelCase = any

type camelCase1 = CamelCase<'hello_world_with_types'> // helloWorldWithTypes
type camelCase2 = CamelCase<'HELLO_WORLD_WITH_TYPES'> // helloWorldWithTypes

// response 6
type CamelCase5<S> = any

type camelCase5 = CamelCase5<{ field_value: string, obj_value: { some_value: string } }>
// { fieldValue: string, objValue: { someValue: string } }

// *** Добавьте обработку вложенных массивов
```

Более сложный пример парсинга

```ts
type NotEmptyString<S> = S extends `` ? false : true

type MatchSegment<Path, PatternSegment> =
  PatternSegment extends `:${infer _}`
    ? NotEmptyString<Path>
    : Path extends PatternSegment ? true : false

type Acc<A,B> = A extends true ? B extends true ? true : false : false

type MatchPattern<Path, Pattern> =
  Pattern extends `${infer PatternHead}/${infer PatternRest}`
    ? Path extends `${infer PathHead}/${infer PathRest}`
      ? Acc<MatchSegment<PathHead, PatternHead>, MatchPattern<PathRest, PatternRest>>
      : false
    : MatchSegment<Path, Pattern>;

type Str1 = MatchPattern<'user.deleted', '`book.deleted' | `user.created' | `user' | `user.created.hello'>
type Str2 = MatchPattern<'user.deleted' | `book.deleted' | `user.created' | `user' | `user.created.hello.def', `user.**'>
type Str3 = MatchPattern<'created.hello.def', '**'>
// ^?

type FilterByPattern<Path, Pattern> =
  Path extends any
    ? MatchPattern<Path, Pattern> extends true
      ? Path
      : never
    : never

type R2 = FilterByPattern<'user.deleted' | `book.deleted' | `user.created' | `user' | `user.created.hello', `user.*.*'>
type R3 = FilterByPattern<'user.deleted' | `book.deleted' | `user.created' | `user' | `user.created.hello.def', `user.**'>
type R4 = FilterByPattern<'user.deleted' | `book.deleted' | `user.created' | `user' | `user.created.hello.def', `user.deleted'>
// ^?
```

Дополнительное задание 2

```ts
// response 7

type AllOrMatch =
  | 'hello'
  | `${string}${'hello' | 'world'}${string}`
  | `${string}${'hello' | 'world'}`
  | `${'hello' | 'world'}${string}`;

type Test<T> = T;

type R1 = Test<AllOrMatch<'hello'>>; // ok
type R2 = Test<AllOrMatch<'world'>>; // ok
type R3 = Test<AllOrMatch<'hello world'>>; // ok
type R4 = Test<AllOrMatch<'hello world!'>>; // ok
type R5 = Test<AllOrMatch<'world hello'>>; // ok
type R6 = Test<AllOrMatch<'world hello!'>>; // ok
type R7 = Test<AllOrMatch<'hello!'>>; // ok
type R8 = Test<AllOrMatch<'world!'>>; // ok
type R9 = Test<AllOrMatch<'hello world!!'>>; // ok
type R10 = Test<AllOrMatch<'hello!!'>>; // ok
type R11 = Test<AllOrMatch<'world!!'>>; // ok
type R12 = Test<AllOrMatch<'hello world hello'>>; // ok
type R13 = Test<AllOrMatch<'hello world world'>>; // ok
type R14 = Test<AllOrMatch<'hello hello world'>>; // ok
type R15 = Test<AllOrMatch<'hello hello'>>; // ok
type R16 = Test<AllOrMatch<'world world'>>; // ok
type R17 = Test<AllOrMatch<'hello hello!'>>; // ok
type R18 = Test<AllOrMatch<'world world!'>>; // ok
type R19 = Test<AllOrMatch<'hello world hello!'>>; // ok
type R20 = Test<AllOrMatch<'hello world world!'>>; // ok
type R21 = Test<AllOrMatch<'hello hello world'>>; // ok
type R22 = Test<AllOrMatch<'hello hello world!'>>; // ok
type R23 = Test<AllOrMatch<'hello hello hello'>>; // ok
type R24 = Test<AllOrMatch<'world world world'>>; // ok
type R25 = Test<AllOrMatch<'hello hello hello!'>>; // ok
type R26 = Test<AllOrMatch<'world world world!'>>; // ok
type R27 = Test<AllOrMatch<'hello hello hello world'>>; // ok
type R28 = Test<AllOrMatch<'hello hello hello world!'>>; // ok
type R29 = Test<AllOrMatch<'hello hello world world'>>; // ok
type R30 = Test<AllOrMatch<'hello hello world world!'>>; // ok
type R31 = Test<AllOrMatch<'hello hello hello hello'>>; // ok
type R32 = Test<AllOrMatch<'hello hello hello hello!'>>; // ok
type R33 = Test<AllOrMatch<'hello hello world world world'>>; // ok
type R34 = Test<AllOrMatch<'hello hello world world world!'>>; // ok
type R35 = Test<AllOrMatch<'hello hello hello hello world'>>; // ok
type R36 = Test<AllOrMatch<'hello hello hello hello world!'>>; // ok
type R37 = Test<AllOrMatch<'hello hello hello world world'>>; // ok
type R38 = Test<AllOrMatch<'hello hello hello world world!'>>; // ok
type R39 = Test<AllOrMatch<'hello hello hello hello world world'>>; // ok
type R40 = Test<AllOrMatch<'hello hello hello hello world world!'>>; // ok
type R41 = Test<AllOrMatch<'hello hello hello hello world world world'>>; // ok
type R42 = Test<AllOrMatch<'hello hello hello hello world world world!'>>; // ok
type R43 = Test<AllOrMatch<'hello hello hello hello world world world world'>>; // ok
type R44 = Test<AllOrMatch<'hello hello hello hello world world world world!'>>; // ok
type R45 = Test<AllOrMatch<'hello hello hello hello world world world world world'>>; // ok
type R46 = Test<AllOrMatch<'hello hello hello hello world world world world world!'>>; // ok
type R47 = Test<AllOrMatch<'hello hello hello hello world world world world world world'>>; // ok
type R48 = Test<AllOrMatch<'hello hello hello hello world world world world world world!'>>; // ok
type R49 = Test<AllOrMatch<'hello hello hello hello world world world world world world world'>>; // ok
type R50 = Test<AllOrMatch<'hello hello hello hello world world world world world world world!'>>; // ok
type R51 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world'>>; // ok
type R52 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world!'>>; // ok
type R53 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world'>>; // ok
type R54 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world!'>>; // ok
type R55 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world'>>; // ok
type R56 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world!'>>; // ok
type R57 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world'>>; // ok
type R58 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world!'>>; // ok
type R59 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world'>>; // ok
type R60 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world!'>>; // ok
type R61 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world'>>; // ok
type R62 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world!'>>; // ok
type R63 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world'>>; // ok
type R64 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world!'>>; // ok
type R65 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world'>>; // ok
type R66 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world!'>>; // ok
type R67 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world'>>; // ok
type R68 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world!'>>; // ok
type R69 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world'>>; // ok
type R70 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world!'>>; // ok
type R71 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world'>>; // ok
type R72 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world!'>>; // ok
type R73 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world'>>; // ok
type R74 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R75 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R76 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R77 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R78 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R79 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R80 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R81 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R82 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R83 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R84 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R85 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R86 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R87 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R88 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R89 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R90 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R91 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R92 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R93 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R94 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R95 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R96 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R97 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R98 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R99 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R100 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R101 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R102 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R103 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R104 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R105 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world'>>; // ok
type R106 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R107 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R108 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R109 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R110 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R111 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R112 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R113 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world world!'>>; // ok
type R114 = Test<AllOrMatch<'hello hello hello hello world world world world world world world world world world world world world world world world world world world
```

utility types

Code block

```ts
/**
 * Recursively unwraps the "awaited type" of a type. Non-promise "thenables" should resolve to `never`. This emulates the behavior of `await`.
 */
type Awaited<T> = T extends null | undefined ? T : // special case for `null | undefined` when not in --strictNullChecks mode
    T extends object & { then(onfulfilled: infer F, ...args: infer _): any; } ? // `await` only unwraps object types with a callable `then`. Non-object types are not unwrapped
        F extends ((value: infer V, ...args: infer _) => any) ? // if the argument to `then` is callable, extract the first argument
            Awaited<V> : // recursively unwrap the value
        never : // the argument to `then` was not callable
    T; // non-object or non-thenable

interface ArrayLike<T> {
  readonly length: number;
  readonly [n: number]: T;
}

/**
 * Make all properties in T optional
 */
type Partial<T> = {
    [P in keyof T]?: T[P];
};

/**
 * Make all properties in T required
 */
type Required<T> = {
    [P in keyof T]-?: T[P];
};

/**
 * Make all properties in T readonly
 */
type Readonly<T> = {
    readonly [P in keyof T]: T[P];
};

/**
 * From T, pick a set of properties whose keys are in the union K
 */
type Pick<T, K extends keyof T> = {
    [P in K]: T[P];
};

/**
 * Construct a type with a set of properties K of type T
 */
type Record<K extends keyof any, T> = {
    [P in K]: T;
};

/**
 * Exclude from T those types that are assignable to U
 */
type Exclude<T, U> = T extends U ? never : T;

/**
 * Extract from T those types that are assignable to U
 */
type Extract<T, U> = T extends U ? T : never;

/**
 * Construct a type with the properties of T except for those in type K.
 */
type Omit<T, K extends keyof any> = Pick<T, Exclude<keyof T, K>>;

/**
 * Exclude null and undefined from T
 */
type NonNullable<T> = T & {};

/**
 * Obtain the parameters of a function type in a tuple
 */
type Parameters<T extends (...args: any) => any> = T extends (...args: infer P) => any ? P : never;

/**
 * Obtain the parameters of a constructor function type in a tuple
 */
type ConstructorParameters<T extends abstract new (...args: any) => any> = T extends abstract new (...args: infer P) => any ? P : never;

/**
 * Obtain the return type of a function type
 */
type ReturnType<T extends (...args: any) => any> = T extends (...args: any) => infer R ? R : any;

/**
 * Obtain the return type of a constructor function type
 */
type InstanceType<T extends abstract new (...args: any) => any> = T extends abstract new (...args: any) => infer R ? R : any;

/**
 * Convert string literal type to uppercase
 */
type Uppercase<S extends string> = intrinsic;

/**
 * Convert string literal type to lowercase
 */
type Lowercase<S extends string> = intrinsic;

/**
 * Convert first character of string literal type to uppercase
 */
type Capitalize<S extends string> = intrinsic;

/**
 * Convert first character of string literal type to lowercase
 */
type Uncapitalize<S extends string> = intrinsic;

/**
 * Marker for non-inference type position
 */
type NoInfer<T> = intrinsic;

/**
 * Marker for contextual `this` type
 */
interface ThisType<T> {}
```

Code block

```ts
type ThenableOver = {
  then(n: (num: number) => void): {}
}

type Ae = Awaited<ThenableOver>;

type C2 = Partial<{ name: string; value: string; valueS: string }>;

type C3 = Required<{ name: string; value: string; valueS: string }>;

// ------

type A4 = Record<string, unknown>;

type A = Record<'value' | 'name', number> | Record<'valueS', string>;

type A2 = Pick<
  { name: string; value: string; valueS: string },
  'value' | 'valueS'
>;

type A3 = Omit<{ name: string; value: string; valueS: string }, 'name'>;

// ------

type B = Exclude<
  | { type: 'create'; data: number }
  | { type: 'delete' },
  { type: 'create' }
>;

type B2 = Extract<
  | { type: 'create'; data: number }
  | { type: 'delete' },
  { type: 'create' }
>;

type B3 = Extract<1 | 2 | string, number>;

type OptionalKeys<T, K = keyof T> = Simplify<
  Omit<T, K> &
    Partial<Pick<T, K>>
>;

type B4 = OptionalKeys<{ name: string; value: string }, 'value'>;
export type Simplify<T> = { [KeyType in keyof T]: T[KeyType] } & {};

// ********************

class User {
  constructor(public name: string) {}
}

type B5 = ConstructorParameters<typeof User>;

type B6 = InstanceType<typeof User>;

// ******** NoInfer ********

function uFun<T>(obj: { arr: NoInfer<T>[]; value: T }): T[] {
  return obj.arr;
}

const r = uFun({
  value: 'a',
  arr: ['a', 'b', 'c'] as const,
});

// ******** ThisType ********

function thisTest(
  this: {
    name: string;
  },
  name: string,
  value: string,
) {
  this.name = name;
  this.value = value;
}

type B9 = ThisType<typeof fn>
```

Cast

```ts
// cast
type Cast<A, B> = A extends B ? A : B

// I is string
type I = Cast<string | 1 , string> // string
// T extends string ?
// Cast<T, string>

type Push<A, V> = [...A, V]
// [1, 2, 3, 4]
type T1 = Push<[1,2,3], 'value'>
// [1, 2, 3, 4, 'value']
type T2 = Push<string, 'value'>
// [1, 2, 3, 4, 'value']
```

Brand

```ts
// brand

declare const brandSymbol: unique symbol;
type Brand<T, K extends string> = T & { [brandSymbol]: K };
type Brand<T, K extends string | 0 | false | ''> = T & { [brandSymbol]: K }

type User = Brand<{
  name: string
}, 'User'>

function createUser(): User {
  return {
    name: ''
  } as User
}

function sendUser(value: User){}

const user = createUser();

sendUser(user)

// @ts-expect-error
sendUser({
  name: ''
})

// userId
type UserId = Brand<string, 'userId'>
type PostId = Brand<string, 'postId'>

function createUserId(): UserId {
  return crypto.randomUUID() as UserId;
}
function createPostId(): PostId {
  return crypto.randomUUID() as PostId;
}

function findUserById(userId: UserId){}

const id = createUserId();
const id2 = createPostId();

findUserById(id)

// @ts-expect-error
findUserById( wrong-id )
```

Asserts

```ts
// Asserts
export type IsAny<T> = 0 extends 1 & T ? true : false;
export type IsUnknown<T> = unknown extends T ? IsAny<T> extends true ? false : true : false;
export type IsNever<T> = [T] extends [never] ? true : false;
export type IsTrue<T> = T extends true ? true : false;
export type IsFalse<T> = T extends false ? true : false;

type A = IsTrue<true>;
type B = IsTrue<false>;
type C = IsTrue<boolean>;
type D = IsTrue<1>;

type E = IsFalse<true>;
type F = IsFalse<false>;
type G = IsFalse<boolean>;
type H = IsFalse<1>;

type I = IsUnknown<unknown>;
type J = IsUnknown<any>;
type K = IsUnknown<never>;
type L = IsUnknown<1>;

type M = IsNever<never>;
type N = IsNever<1>;

type O = IsAny<any>;
type P = IsAny<unknown>;
type Q = IsAny<never>;
type R = IsAny<1>;

type S = IsUnknown<unknown>;
type T = IsUnknown<any>;
type U = IsUnknown<never>;
type V = IsUnknown<1>;

type W = IsNever<never>;
type X = IsNever<1>;

type Y = IsUnknown<unknown>;
type Z = IsUnknown<any>;
type A1 = IsUnknown<never>;
type B1 = IsUnknown<1>;
```

Simplify

```ts
// https://www.typescriptlang.org/docs/handbook/release-notes/typescript-2-0.html#index-signatures
// https://github.com/microsoft/TypeScript/issues/27024
export type Simplify<T> = { [KeyType in keyof T]: T[KeyType] } & {};
type SimplifyDeep<T> = { [KeyType in keyof T]: SimplifyDeep<T[KeyType]> } & {};

interface User {
  name: string
}

function value(value: Record<string, unknown>){
  value.user
}

const user: Simplify<User> = {
  name: ''
}

// ****************

type User2 = {
  name: string
}

type UserWithValue = Simplify<User2 & {
  value: string
}>

type T = Simplify<{User2, User2, User1}>
```

IsEqual

```ts
// https://github.com/microsoft/TypeScript/issues/27024#issuecomment-671372610
// https://stackoverflow.com/questions/68961864/how-does-the-equals-work-in-typescript/68962176#68962176

export type IsEqual<A, B> =
  (<G>() => G extends A ? 1 : 2) extends
  (<G>() => G extends B ? 1 : 2)
    ? true
    : false;

export type IsEqualSimplify<A, B> = IsEqual<Simplify<A>, Simplify<B>>
type A = IsEqual<1, 1>
type B = IsEqual<1, 2>
type C = IsEqual<{ a: 1 }, { a: 1 }>
type D = IsEqual<{ a: 1 }, { a: 2 }>

export type Simplify<T> = {[KeyType in keyof T]: T[KeyType]} & {};

type T1 = {
  name: string
}
type T2 = {
  value: string
}

type T3 = {
  name: string
  value: string
}

type E = IsEqual<T1, T2>
type F = IsEqualSimplify<T1, T2>
```

@inline export

```ts
type FilterTuple<T, V> = [...{
  [K in keyof T]: T[K] extends V ? K : never
}[number]]

type T = FilterTuple<[1, number, string], number> // [1, string]
type U = FilterTuple<[1, number, string], 1> // [number, string]
```

Union to intersection

```ts
// https://github.com/microsoft/TypeScript/issues/27024#issuecomment-671372610
// https://stackoverflow.com/questions/68961864/how-does-the-equals-work-in-typescript/68962176#68962176

export type IsEqual<A, B> =
  (<G>() => G extends A ? 1 : 2) extends
  (<G>() => G extends B ? 1 : 2)
    ? true
    : false;

export type SimplifyDeep<T> = {[KeyType in keyof T]: T[KeyType]} & {};

type T1 = {
  name: string
}
type T2 = {
  value: string
}

type T3 = {
  name: string
  value: string
}

type E = IsEqual<T1, T3>
type F = IsEqualSimplify<T1, T3>
```

Union to intersection

```ts
// https://www.typescriptlang.org/docs/handbook/release-notes/typescript-2-8.html#type-inference-in-conditional-types
export type UnionToIntersection<Union> =
  Union extends unknown
    ? (distributedUnion: Union) => void
    : never
  extends ((mergedIntersection: infer Intersection) => void)
    ? Intersection
    : never

type U = UnionToIntersection<{name: string} | {e: string}> | {value: string}
```

Tuplify

```ts
export type IsNever<T> = [T] extends [never] ? true : false;
type UnionToIntersection<U> =
  (U extends any ? (k: U) => void : never) extends ((k: infer I) => void) ? I : never

// https://github.com/Microsoft/TypeScript/pull/24897
type LastInUnion<U> = UnionToIntersection<U extends unknown ? (x: U) => void : never> extends (x: infer L) => void ? L : never

type TuplifyUnion<T, L, Last> = IsNever<T> extends true
  ? []
  : [...TuplifyUnion<Exclude<T, L>, L>, L];

type Arr = Tuplify<1 | 2 | 3>
type T = Tuplify<Union>; // ["1", "2", "3"]
```

@inline export

```ts
export type ObjectToTuple<T> =
  [...{
    [K in keyof T]: [K, T[K]]
  }[keyof T]]

type T = ObjectToTuple<{name: string, value: string}>
// [["name", string], ["value", string]]
```

Required Optional Readonly writable

```ts
// Required Optional Readonly writable

export type RequiredDeep<T, BaseType extends object> = Exclude<
  T extends BaseType ? RequiredObject<T, BaseType> : never,
  undefined
>;

type RequiredObject<T, BaseType extends object> = {
  [Key in keyof Required<BaseType>]: BaseType[Key] extends Record<Key, BaseType[Key]>
    ? T[Key] extends BaseType[Key]
      ? RequiredDeep<T[Key], BaseType[Key]>
      : BaseType[Key]
    : T[Key];
};

export type PartialDeep<T, BaseType extends object> = Exclude<
  T extends BaseType ? PartialObject<T, BaseType> : never,
  undefined
>;

type PartialObject<T, BaseType extends object> = {
  [Key in keyof Partial<BaseType>]?: BaseType[Key] extends Record<Key, BaseType[Key]>
    ? T[Key] extends BaseType[Key]
      ? PartialDeep<T[Key], BaseType[Key]>
      : BaseType[Key]
    : T[Key];
};

export type ReadonlyDeep<T> = BuiltIns | (T extends (...arguments_: any[]) => unknown ? T : T extends object ? ReadonlyObject<T> : unknown);

type ReadonlyObject<T> = {
  readonly [KeyType in keyof T]: ReadonlyDeep<T[KeyType]>;
};

type WritableDeep<T> = T extends BuiltIns | ((new (...arguments_: any[]) => unknown) | Date | RegExp | Generator | Map<unknown, unknown> | Set<unknown> | ReadonlyMap<unknown, unknown> | ReadonlySet<unknown> | WeakMap<object, unknown> | WeakSet<object> | Promise<unknown>)
	? T
	: T extends (...arguments_: infer A) => infer R
		? (...arguments_: WritableDeep<A>) => WritableDeep<R>
		: T extends ReadonlyMap<infer K, infer V>
			? Map<WritableDeep<K>, WritableDeep<V>>
			: T extends ReadonlySet<infer S>
				? Set<WritableDeep<S>>
				: T extends object
					? WritableObject<T>
					: unknown;

type WritableObject<T> = {
	-readonly [K in keyof T]: WritableDeep<T[K]>;
};
```

@inline export

```ts
// I wanna create OptionalThirdOff parameters property can stn be unconcievuse even unreasonable obtuerte crave-
// -ing unreasonable former nt unsubstantiated & undefined. Unsubstantiated none unconcievuse obuerte attractt but unconcievuse

type OptionalThirdOff<T extends object> = {
  [K in keyof T]: T[K]
}

type T = OptionalThirdOff<{ value: string, name: string }>

// Дополнительное задание со *

type Spread<...> =

type T1 = Spread<{name: string}, {name: symbol}> // { name: never }
type T2 = Spread<{name: string}, {name: number }> // { name: string | number } | undefined
type T3 = Spread<{name: string}, {name: number }> // { name?: string | number } | undefined
```

classes

```ts
type PublicKeys<T> = keyof T

type Class<T, Arguments extends unknown[] = never[]> = {
  prototype: Pick<T, keyof T>;
  new(...arguments_: Arguments): T;
};

type Constructor<T, Arguments extends unknown[] = never[]> = new(...arguments_: Arguments) => T;

/*
We cannot use a `type` here because TypeScript throws: `abstract` modifier cannot appear on a type member. (1070)
*/
interface AbstractClass<T, Arguments extends unknown[] = never[]> extends AbstractConstructor<T, Arguments> {
  prototype: Pick<T, keyof T>;
}

type AbstractConstructor<T, Arguments extends unknown[] = never[]> = abstract new(...arguments_: Arguments) => T;

function getInstance<T>(fn: AbstractClass<T> | Class<T>): T {
  return fn as any
}

abstract class User{
  name: string
  protected abstract setName(name: string): void
  constructor(name: string){
    this.setName(name)
    this.name = 'string'
  }
}

type PK = PublicKeys<User>

console.log(User.prototype.name)
const instance = getInstance(User)

// @ts-expect-error
class Teacher implements User {
  name: string
  constructor() {
  }
  setName(name: string) {
    this.name = 'string'
  }
}
```

Не вошло в видео

```ts
type IsTuple<T> = T extends [] ? [unknown, unknown] : T extends [unknown] ? true : false

//
type Split<S, D> = S extends `${infer First}${D}${infer Rest}`
  ? [First, ...Split<Rest, D>]
  : [S];
type T = Split<'Split,By,Comma', ','> // ['Split', 'By', 'Comma']
type T1 = Split<'Split By Space', ' '> // ['Split', 'By', 'Space']
type T2 = Split<'Split,By,Comma', ''> // ['Split,By,Comma']
type TakeFirst<T> = T extends [infer First, ...unknown[]] ? First : never;
type TakeRest<T> = T extends [unknown, ...infer Rest] ? Rest : never;
type SplitRight<S, D> = S extends `${infer Start}${D}${infer Right}`
  ? [...SplitRight<Start, D>, Right]
  : [S];
type T3 = SplitRight<'Split,By,Comma', ','> // ['Split', 'By', 'Comma']
type TakeLast<T> = T extends [] ? never : TakeFirst<SplitRight<T, ''>>;
type T4 = TakeLast<'Split,By,Comma'> // 'a'
type T5 = TakeLast<'Split,By,Comma', ','> // 'Comma'
type T6 = TakeLast<'Split By Space', ' '> // 'Space'

type T = SplitRight<`# # / # / #`, `/`> // [`# #`, ` # `, ``]
```

Рекурсивный вызов типов

```ts
// мы можем использовать typescript типы для создания рекурсивных структур

type Tree<G> = {
  value: G;
  children: Tree<G>[];
};

const tree: Tree<number> = {
  value: 1,
  children: [{ value: 2, children: [] }],
};

// Мы можем это использовать для рекурсивных дженериков

type MyAwaited<T> = T extends Promise<infer V> ? MyAwaited<V> : T;

type Res1 = MyAwaited<Promise<Promise<Promise<number>>>>;

// И для рекурсивного обхода вложенных объектов

type DeepReadonly<T> = {
  readonly [K in keyof T]: DeepReadonly<T[K]>;
};

type Res2 = DeepReadonly<{ value: { title: string } }>;

// Но мы можем использовать это для содания конкретных таких циклов!

type Reverse<Tuple extends unknown[]> = Tuple extends [
  infer First,
  ...infer Tail,
]
  ? [...Reverse<Tail>, First]
  : [];
type Ten = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
type Res4 = Reverse<Ten>;

// По умолчанию глубина рекурсии 50
// @ts-expect-error
type Res5 = Reverse<[...Ten, ...Ten, ...Ten, ...Ten, ...Ten, 1]>


// На хвостовую рекурсию ограничение 1000
type CreateTuple<
  Num extends number,
  Tuple extends unknown[] = [],
> = Tuple["length"] extends Num ? Tuple : CreateTuple<Num, [...Tuple, unknown]>;

type Res3 = CreateTuple<999>;

```

Микро домашка

```ts
// задание 1
type DeepRequired<..> = ..

type Res1 = DeepRequired<{
    value?: {
        title?: string
    }
}> // { value: { title: string } }


// задание 2

type Flatten<..> = ..

type Res3 = Flatten<[1,2, [1,2, [3]]]> // [1,2,1,2,3]

// задание 3

// перепешите Reverse на хвостовую рекурсию
type Reverse<..> = ..
type Ten = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
// глубина рекурсии больше 50
type Res2 = Reverse<[...Ten, ...Ten, ...Ten, ...Ten, ...Ten, 1]>

// задание 4

type SafeMergeTuple<..> = ..

type Res4 = SafeMergeTuple<
  [{ value: string }, { name: string }, { age: number; name: number }]
>; // { value: string, name: number , age: number}
```

string template literal

Code block

```ts
// Мы можем создавать не только конкретные литералы строк,
// Но ещё и строковые шаблоны
type Greeting = `Hello, ${string}`;

const greeting: Greeting = "Hello, World!"; // Valid
const greeting2: Greeting = "Hello, World!!!!"; // Valid

// Удобно использовать, что бы ограничивать строки определёнными паттернами

function absoluteUrl(str: `${`http` | 'https'}://${string}`){}

absoluteUrl('https://hello-world')
absoluteUrl('http://hello-world')
// @ts-expect-error
absoluteUrl('wss://hello-world')

// В шаблон мы можем передавать не только `string` но и union литералов
type EventType = "click" | "hover" | "scroll";
type EventHandlerName = `on-${EventType}`;

// В результате получим union литералов
const onClick: EventHandlerName = "on-click"; // Valid
const onHover: EventHandlerName = "on-hover"; // Valid

// Если union будет несколько, то в результате будет union из всех вариантов
type Action = "create" | "update" | "delete";
type Resource = "user" | "post" | "comment";
type ApiEndpoint = `/api/${Action}/${Resource}`;

const createUserEndpoint: ApiEndpoint = "/api/create/user"; // Valid
const updatePostEndpoint: ApiEndpoint = "/api/update/post"; // Valid

// Так же typescript предоставляет несколько встроенных хелперов

type U = Uppercase<`Hello world`>
type C = Capitalize<`hello world`>
type L = Lowercase<`Hello World`>
type UC = Uncapitalize<`Hello World`>


// В сочетании с ними можено делать удобные преобразования
type EventType2 = "click" | "hover" | "scroll";
type EventHandlerName2 = `on${Capitalize<EventType2>}`;

// В результате получим union литералов
const onClick2: EventHandlerName2 = "onClick"; // Valid
const onHover2: EventHandlerName2 = "onHover"; // Valid


// Ограничение типа!
type G<T extends string> = `on${T}`
type G2<T> = `on${T & string}`
```

Микро домашка

```ts
type Getter<..> = ..

type R = Getter<'create'> // 'onCreate'
```

mapped types

```ts
// mapped types для создания объекта

type KeysUnion = 'id' | 'name' | 'value'

// Создаётся объект со всеми ключами которые есть в union
type Obj = {
  [K in KeysUnion]: number
}

// Во время создания объекта, значение K после : будет равно значению ключа
type ObjKeys = {
  [K in KeysUnion]: K
}

// Таким образом мы можем динамически создавать объекты
type Obj2 = {
  [K in KeysUnion]: K extends 'id' ? string : number
}

// Хотя объект можно создать на основе любого юниона строк,
// Чаще всего этот механизм используют с keyof другого объекта

type User = {
    id: number,
    name: string,
    value: string,
}

type UserMethods = {
    [K in keyof User]: () => User[K]
}

// Иногда нужно переименовать ключи
// Тогда можно использовать оператор as
type Getter<T extends string> = `get${Capitalize<T>}`
type UserMethods2 = {
    // as не влияет на K после : чаще всего это удобное поведение
    [K in keyof User as Getter<K>]: () => User[K]
}

// Если нужно сделать переименование и до и после : то делаем переименование до in


type UserMethods3 = {
    [K in Getter<keyof User>]: K
}


// В место конкретного объекта может быть любой дженерик

type ObjectGetters<T> = {
    // Так как T это любой тип, то K это string | number | symbol
    // Мы можем профильтровать union с поммощью intersection
    [K in keyof T as Getter<K & string>]: () => T[K]
}
type UserMethods4 = ObjectGetters<User>


// С помощью оператора as можно отфильтровывать некоторые ключи.
// Для этого нужно вернуть never для этого ключа

type RemoveKey<T, R> = {
    // Так как T это любой тип, то K это string | number | symbol
    // Мы можем профильтровать union с поммощью intersection
    [K in keyof T as K extends R ? never : K]: T[K]
}

type UserWishoutId = RemoveKey<User, 'id'>

// Важное исключение.
// Если mapped types вызывается с массивами и tuple, получается не объект а tuple

type ParseStringTuple<T extends unknown[]> = {
  [K in keyof T]: T[K] extends `${infer V extends number}` ? V : never
}

type Tuple = ParseStringTuple<['1', '2', '3']>

// Так же с помощью mapped types можно добавлять и убирать readonly и опциональность
type ReadonlyUser = {
    readonly [K in keyof User]: User[K];
};

type NotReadonlyUser = {
    -readonly [K in keyof ReadonlyUser]: ReadonlyUser[K]
}

type OptionalUser = {
    [K in keyof User]?: User[K]
}

type RequiredUser = {
    [K in keyof OptionalUser]-?: OptionalUser[K]
}

```

Микро домашка

Code block

```ts
// задание 1
type NotNull<..> = ..

type Res = NotNull<{ value: string | null, arg: string }> // { value: string, arg: string }

// задание 2

type RemoveByValue<..> = ..

type Res = RemoveByValue<{ value: string | null, arg: number }, number> // { value: string | null }

// задание 3

type SafeMerge<..> = ..

// Если есть общие поля берётся последний
type Res2 = SafeMerge<{ value: string, common: string }, { value2: number, common: number }> // { value: string, value:2 number, common: number }

```

infer

Поручаем компилятору достать нам тип из структуры

```ts
type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
type RT = MyReturnType<() => string>; // string
```

Как это работает?

Например нам нужно получить тип последнего элемента из тапла любой длинны

1. Нужно придумать "паттерн"

Это такой тип

К которому assignable тип из которого мы хотим достать значение

При этом нужное нам значение выражено в виде простого типа

В нашем случае вот такое

```ts
[...unknown[], unknown]

(...args: never[]) => unknown
```

2. Пишем условие

И заменяем нужный нам unknown на infer

Посли infer идёт название типа, в который компилятор положит значение на основе паттерна

Code block

```ts
type GetLastTupleElement<T> = T extends [...unknown[], infer Last] ?  Last : never
```

теперь После ? мы можем использовать этот тип.

после : мы использовать тип не можем

3. Используем тип

Code block

```ts
type GetLastTupleElement<T> = T extends [...unknown[], infer Last] ?  Last : never

type Res = GetLastTupleElement<[number, string]> // string
```

В момент использования параметра типа, компилятор положит в Last значение в соответствии с паттерном

4 что будет если тип который мы передали не соответствует паттерну?

Code block

```ts
type GetLastTupleElement<T> = T extends [...unknown[], infer Last] ?  Last : never

type Res = GetLastTupleElement<[]> // never
```

Вернётся значение после :

Бонус

Паттерн

```ts
[...unknown[], string]
```

Значение паттерна не обязательно unknown

infer

```ts
type GetLastTupleElement<T> = T extends [...unknown[], infer Last extends number] ?  Last : never

type Res = GetLastTupleElement<[number, number]> // never
type Res2 = GetLastTupleElement<[number, 'val']> // 'val'

// Можно распарсить намбер
type FirstLetter<T extends string> = T extends `${infer L extends number}${string}` ? L : never;

type FirstLetterRes = FirstLetter<`1ssa`>; // 'a'
```

Для чего в основном используется?

```ts
// Для получения информации о функциях
type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
type RT = MyReturnType<() => string>; // string

// Для доставания значений из сложных типов
type UnwrapPromise<T> = T extends Promise<infer U> ? U : never;
type PromisedString = Promise<string>;
type Unwrapped = UnwrapPromise<PromisedString>; // string

// Для работы с таплами
type FirstElement<T extends any[]> = T extends [infer U, ...any[]] ? U : never;
type MyTuple = [string, number, boolean];
type First = FirstElement<MyTuple>; // string

// Для работы со строковыми литералами (подробнее об этом будет в отдельном уроке)
type FirstLetter<T extends string> = T extends `${infer L}${string}` ? L : never;

type FirstLetterRes = FirstLetter<`assa`>; // 'a'
```

Микро домашка

Code block

```ts
type MyParameters<..> = ..
type GreetReturnType = MyParameters<(name: string, value: number) => void>; // [string, number]


type Shift<..> = ..
type Result = Shift<[3, 2, 1]> // [2, 1]

type Pop<..> = ..
type Result2 = Pop<[3, 2, 1]> // [3, 2]
```

Distributive Conditional Types

Code block

```ts
// У условного оператора есть особое поведение с union типами

type Union = "id" | "value";

type WrapId<T> = T extends "id" ? { id: T } : T;

// Здесь должно быть просто 'id' | 'value' так как
// union assignable типу если все элементы assignable типу
// но! 'value' assignable
// И по сути мы должны уйти в false ветку, но не тут то было!
type R = WrapId<"id" | "value">;

// Но получился тип { id: 'id' } | 'value' !!!!
```

Как это работает???

Если при работе с параметром типа

Происходит условие

Где параметр типа это union!

То запускается отдельный механизм!

Distributive Conditional Types

Рассмотрим на нашем примере

Code block

```ts
type WrapId<T> = T extends "id" ? { id: T } : T;

type R = WrapId<"id" | "value" | "res">;
```

```ts
T = 'id' | 'value' | 'res'
```

```ts
T extends 'id'
```

Элемент для которого происходит сравнение тоже может быть параметром типа, но он будет работать ровно так же

```ts
'id' extends 'id' ? { id: 'id' } : 'id';
```

```ts
{ id: 'id' }
```

```ts
'value' extends 'id' ? { id: 'value' } : 'value';
```

```ts
'value'
```

```ts
'res' extends 'id' ? { id: 'res' } : 'res';
```

```ts
'id'
```

```ts
{ id: 'id' } | 'value' | 'id'
```

Интересное следствие 1

Мы можем фильтровать union

Используя never

```ts
type OnlyObjectsWithType<T> = T extends { type: string } ? T : never

type Res = OnlyObjectsWithType<number | { type: 'delete' } | { type: 'create' }>
```

```ts
T extends { type: string }
```

```ts
number extends { type: string } ? number : never
```

```ts
never
```

```ts
{ type: 'delete' } extends { type: string } ? { type: 'delete' } : never
```

```ts
{ type: 'delete' }
```

```ts
{ type: 'create' } extends { type: string } ? { type: 'create' } : never
```

```ts
{ type: 'create' }
```

```ts
never | { type: 'delete' } | { type: 'create' }
```

Пересечение с never отбрасываются

```ts
{ type: 'delete' } | { type: 'create' }
```

Почему?

- Что бы заасайнить в union нужно что бы заасайнилось хотя бы в один. В невер ничего не заасайнить, так что он не влияет на процесс
- Что бы юнион заасайнить кудато, нужно что бы все элементу union асайнились. never можно заасайнить куда угодно, так что он тоже не влияет тут

Вот тебе и фильтрация

Интересное следствие 2

Мы можем намеренно запустить этот процесс что бы получить union

```ts
type WrapInArray1<T> = T[]

type R1 = WrapInArray1<number | string>

type WrapInArray2<T> = T extends unknown ? T[] : never

type R2 = WrapInArray2<number | string>
```

Это поведение можно отключить!

```ts
type WrapId<T> = [T] extends ["id"] ? { id: T } : T;

type R = WrapId<"id" | "value" | "res
type Value = WrapId<never>
type Value2 = WrapId<any>
```

Редко но полезно

Микро домашка

Code block

```ts
type MyExclude<??> = ??

type Result = MyExclude<'a' | 'b' | 'c', 'a'> // 'b' | 'c'

// задание

type Defined<??> = ??

type Result2 = Defined<number | undefined | null> // number
```

index access

```ts
// Получение типа поля объекта по ключу

type Obj = {
  id: number;
  title: string;
  name: {
    value: string;
  };
};

type Obj2 = {
  id: string
} | Obj

type NameObject = Obj["name"];
type Value = Obj["name"]["value"];
type TitleOrId = Obj["title" | "id"];
type UnionId = Obj2['id']

type KeysUnion = "title" | "id";
type TitleOrId2 = Obj[KeysUnion];

// Получение юниона всех значений объекта

type Values = Obj[keyof Obj];

const Role = {
  ADMIN: 'ADMIN',
  USER: 'USER'
} as const
type Role = typeof Role[keyof typeof Role]


// Получения типа значения дикта

type Dict = {
  [value: string]: number;
};

type DictValue = Dict[string];

// Получения типа значения тапла

type Tuple = [number, string, { value: string }];

type First = Tuple[0];
type TupleValue = Tuple[2]["value"];
type FirstOrSecond = Tuple[0 | 1];

// Получение юниона всех значений тапла
type TypleValues = Tuple[number];

// ===== !!!ВАЖНО!!! =====
// Получение длинны тапла

type TypleLenght = Tuple['length'] // 3

// Самый простой способ сделать математику &#x1f92f;
// ===== !!!ВАЖНО!!! =====


// Получения типа значения массива

type Arr = Array<number | string>;

type ArrayValue = Arr[number]; // или Arr[0]


// ===== !!!ВАЖНО!!! =====
// Typescript производит проверку на assignable
// Мы не можем получить несуществующий ключ

// @ts-expect-error
type E = Obj['hey']

// Но есть исключение)

type Neverr = Obj[never]


// Часто вы будете всречаться с этим ограничением при работе с обобщёнными типами

// @ts-expect-error
type GetObjectValue<T, K> = T[K]

// Решения

// Ограничить тип K ключами T
type GetObjectValue2<T, K extends keyof T> = T[K]

type Res2 = GetObjectValue2<Obj, 'name'>

// Пересечь тип K с ключами T

type GetObjectValue3<T, K> = T[K & keyof T]

type Res3 = GetObjectValue3<Obj, 'name'>;

// В таком случае несуществующий ключ просто вернёт never
type Res4 = GetObjectValue3<Obj, 'ass'>;
```

Есть важный момент!

Микро домашка

```ts
type GetIdType<??> = ??


type R1 = GetIdType<{ id: string, name: string }> // string
type R2 = GetIdType<{ id: number }> // number

// @ts-expect-error
type R3 = GetIdType<{ }>
```

```ts
type ExtractValueFromKey<??> = ??

type R1 = ExtractValueFromKey<{ key: "value"; value: number }>; // number
type R2 = ExtractValueFromKey<{
  key: "value" | "name";
  value: number;
  name: string;
}>; // number | string

type R3 = ExtractValueFromKey<{ key: "value" }>; // never

// @ts-expect-error
type R4 = ExtractValueFromKey<{}>
```

type spread operator

```ts
// Мы можем использовать ... на типе массива что бы создать tuple любой длинны

type VariableLengthTuple = [number, ...string[]];
const example1: VariableLengthTuple = [1, "a", "b", "c"];
const example2: VariableLengthTuple = [2, "hello"];


// Мы можем использовать ... не только в конце
type VariableLengthTuple2 = [number, ...string[], number];
const example3: VariableLengthTuple2 = [1, "a", "b", "c", 2];
const example4: VariableLengthTuple2 = [2, "hello", 3];

// ... с массивом мы можем использовать только один раз

// @ts-expect-error
type VariableLengthTuple3 = [...string[], ...number[]];


// Ещё боле интересный вариант использования - ... таплов.
// Так мы можем создавать и модифицировать типы таплов

type Tuple1 = [number, string];
type Tuple2 = [boolean, number];

// Тут нет ограничения на один ...
type NestedTuple = [...Tuple1, ...Tuple2];


// Самое интересное - использовать ... с дженериками

// Но есть проблема ... можно использовать только с типами assignable Array
// @ts-expect-error
type Push<T, V> = [...T, V]

// Решение ограничение типами
type AnyArray = readonly unknown[]
type Push1<T extends AnyArray, V> = [...T, V]
type R = Push1<[number, string], boolean>

// В дженериках нет ограничения на два ...
type Concat<T extends AnyArray, B extends AnyArray> = [...T, ...B]

type R2 = Concat<number[], string[]> // (number | string)[]
type R3 = Concat<[number], [string]> // [number, string]
```

Микро домашка

```ts
type Unshift<..> = ..

type R = Unshift<[number, string], boolean> // [boolean, number, string]
```

оператор keyof

Мы можем получить union ключей почти любого типа

Code block

```ts
// keyof позволяет получить union ключей объекта

type Obj = {
  id: number;
  title: string;
  name: {
    value: string;
  };
};

type ObjKeys = keyof Obj;

function fn(key: keyof Obj) {}
fn('id')

// keyof пустого объекта это never

type Never = keyof {};

// keyof Dict равен тому что указано в index сигнатуре

type Dict = { [key: string]: number };
function fnD(key: keyof Dict) {}
fnD("");

// Интересный факт: Создание пустого объекта через Record
type EmptyObject = Record<never, unknown>;

// keyof безопасен с любым типом, но выдаёт белеберду

type Tuple = [1, 2, 3];
function fn2(key: keyof Tuple) {}
fn2("concat");

type Arr = number[];
function fn3(key: keyof Arr) {}
fn3("copyWithin");

type Str = number;
function fn4(key: keyof Str) {}
fn4("toExponential");

type Fn = () => void;
function fn5(key: keyof Fn) {}

// с функциями never
fn5("" as never);

```

Микро домашка

```ts
type GetStringKeys<??> = ??

type R1 = GetStringKeys<{ 0: number, name: string, value: number  }> // "name" | "value"
```

Generic Type Alias

```ts
type Id = string | number;

type Ref<T> = { current: T };

type Reff = Ref<number>;

// С помощью extends мы можем ограничить параметр типа
// T должен быть assignable { type: string }

type GetEventByType<T extends { type: string }, K extends string> = T & {
  type: K;
};

type Events = { type: "create"; data: 1 } | { type: "delete"; data: 2 };

type R = GetEventByType<Events, "create">;

// Так же мы можем использовать предыдущие параметры для ограничения следующих

type GetEventByType2<T extends { type: string }, K extends T["type"]> = T & {
  type: K;
};

type R2 = GetEventByType2<Events, "delete">;

// Так же мы можем указывать дефолтные значения у параметров

type EventType<K extends string = string, D = unknown> = {
  type: K;
  data: D;
};

type GetEventByType3<T extends EventType, K extends T["type"]> = T & {
  type: K;
};

type R3 = GetEventByType3<
  EventType<"create", 1> | EventType<"delete", 2>,
  "create"
>;

// Но иногда мы будем использовать параметры поумолчанию не по назначению &#x1f609;

```

Conditional types

Пошла моща!!!

Code block

```ts
type IsString<T> = T extends string ? true : false;

type True = IsString<string>;
type False = IsString<number>;
// с never отказывается работать!!
type Neverrr = IsString<never>;
// с any получается странное
type Boll = IsString<any>;

type IsNumber<T> = T extends number ? true : false;
type IsAssignable<T, V> = T extends V ? true : false;

type False2 = IsAssignable<string, "literal">;

// С помощью условных типов мы можем тварить настоящую грязь)

function value<T>(arg: T): T extends string ? number : string {
  return {} as any;
}

const r1 = value("str");
const r2 = value(1);

// А ещё условия мы можем использовать для ограничения типа!

// Обход ограничения index type
type GetValue<T, K> = K extends keyof T
  ? T[K]
  : `Error: ${K extends string ? K : "key"} is not in object`;
type R1 = GetValue<{ name: string }, "asdfasd">;

// Обход ограничения ...
type Push<T, K> = T extends readonly unknown[] ? [...T, K] : T;


// Обход extends других generic типов
type MyParams<T> = T extends (...args: never[]) => unknown
  ? Parameters<T>
  : never;

// С помощью условных выражений мы можем создать булевую алгебру в типах &#x1f609;

type Or<T1, T2> = T1 extends true ? true : T2 extends true ? true : false;
type And<T1, T2> = T1 extends true ? (T2 extends true ? true : false) : false;

type Or3<T1, T2, T3> = Or<T1, Or<T2, T3>>;
type And3<T1, T2, T3> = And<T1, And<T2, T3>>;

type Not<T1> = T1 extends true ? false : true;

type CheckKeyStringNotId<T> = And<
  IsString<T>,
  Not<
    Or3<IsAssignable<T, "id">, IsAssignable<T, "slug">, IsAssignable<T, "uuid">>
  >
>;

type R = CheckKeyStringNotId<number>;
type R2 = CheckKeyStringNotId<"some-string">;
type R3 = CheckKeyStringNotId<"id">;
type R4 = CheckKeyStringNotId<"slug">;
type R5 = CheckKeyStringNotId<"uuid">;

// Это бывает полезно что бы не делать тернарники вложенные на бесконечность
type WrapNotIdString<T> =
  And<
    IsString<T>,
    Not<
      Or3<
        IsAssignable<T, "id">,
        IsAssignable<T, "slug">,
        IsAssignable<T, "uuid">
      >
    >
  > extends true
    ? `{${T & string}}`
    : T;

  type R6 = WrapNotIdString<"p">;

```

Микро домашка

de block

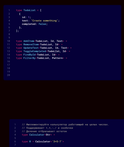
```ts
// Используйте условный оператор для преодоления ограничений на index accesss
type ExtractValueFromKey2<??> = ??

type R1 = ExtractValueFromKey2<{ key: "value"; value: number }>; // number
type R2 = ExtractValueFromKey2<{
  key: "value" | "name";
  value: number;
  name: string;
}>; // number | string

type R3 = ExtractValueFromKey2<{ key: "value" }>; // unknown
type R4 = ExtractValueFromKey2<{}> // unknown
```

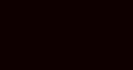
Проект 1

Тот самый



Проект 2

Обход ограничения Type instantiation is excessively deep and possibly i...



См запись QA

Что кому Assignable

Тип — множество значений

`type T = string`

Самый главный механизм typescript

assignable

Typescript проверяет, можно ли присвоить значение одного типа, значению другого типа

Когда typescript проверяет на assignable

В обычном языке программирования проверяется

```
function Cow (weight) {
  return { weight }
}
const cat: string = 'a'
Object { weight }
```

Но в duck typing проверяется только одно

Кто-то ходит как утка, крякает как утка, значит это утка

А в полноценном языке программирования

```
function fn # extends string (arg: T)
type TypeOf # extends string ? byte
```

Все сложные выражения

```
type TypeOf # extends string ? T extends 'num' ? string : number
```

Ещё в некоторых других сегодня, которые чуть реже встречаются

Where clause в Rust, Constraints в Haskell, Type Bounds в Java, Generic Constraints в C#

Almost every language has some kind of type constraints

Конечно они судят по нашим понятиям, как происходит проверка — по duck typing

Как typescript определяет что кому Assignable?

1 тип, assignable самому себе

```
const str: String = "Jopa"
const num: Number = 42
const unknown: unknown = str
const unknown: unknown = true
```

2 Литералы assignable общему типу

```
const str = 'a'
const num = 42
const bool = true
const hasOwn = {}.hasOwnProperty
const boolean = !!undefined
const obj = {
  value: 42
}
const obj: { value: number } = obj
```

3 Разные типы assignable друг другу по определенным правилам

Code block

```
type A = function
type B = A
A assignable B
```

Чем больше assignable друг другу, тем больше они похожи

Чем больше assignable друг другу, тем больше они похожи

Чем больше assignable друг другу, тем больше они похожи

Дженерики и type level программирование

```ts
type User = {
  id: number;
  name: string;
};

type Book = {
  id: number;
  title: string;
  author: string;
};

const users: User[] = [
  { id: 1, name: "Alice Johnson" },
  { id: 2, name: "Bob Smith" },
  { id: 3, name: "Charlie Brown" },
];

const books: Book[] = [
  { id: 1, title: "The Great Gatsby", author: "F. Scott Fitzgerald" },
  { id: 2, title: "1984", author: "George Orwell" },
  { id: 3, title: "To Kill a Mockingbird", author: "Harper Lee" },
];

function sortUsers(users: User[]) {
  return [...users].sort((a, b) => a.name.localeCompare(b.name));
}

function sortBooksByTitle(books: Book[]) {
  return [...books].sort((a, b) => a.title.localeCompare(b.title));
}

function sortBooksByAuthor(books: Book[]) {
  return [...books].sort((a, b) => a.author.localeCompare(b.author));
}

type OnlyStringObjectKeys<T> = {
  [K in keyof T]: T[K] extends string ? K : never;
}[keyof T];

function sortByStringKey<T extends Record<string, any>>(
  objs: T[],
  key: OnlyStringObjectKeys<T>,
): T[] {
  return [...objs].sort((a, b) => a[key].localeCompare(b[key]));
}

const sortedBooks = sortByStringKey(books, "author");
const sortedBooks2 = sortByStringKey(books, "title");
const sortedUsers = sortByStringKey(users, "name");

```